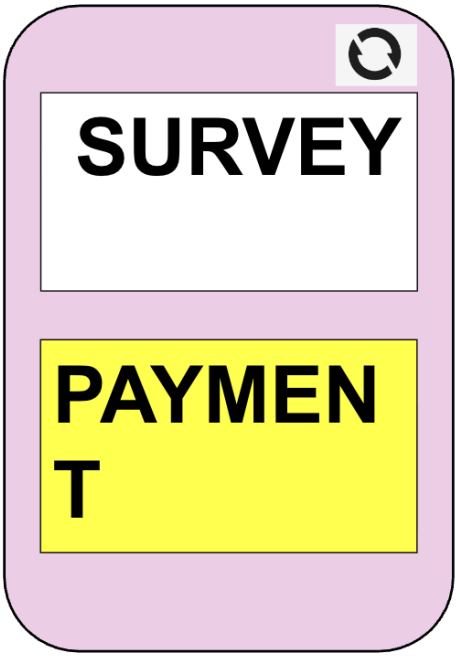
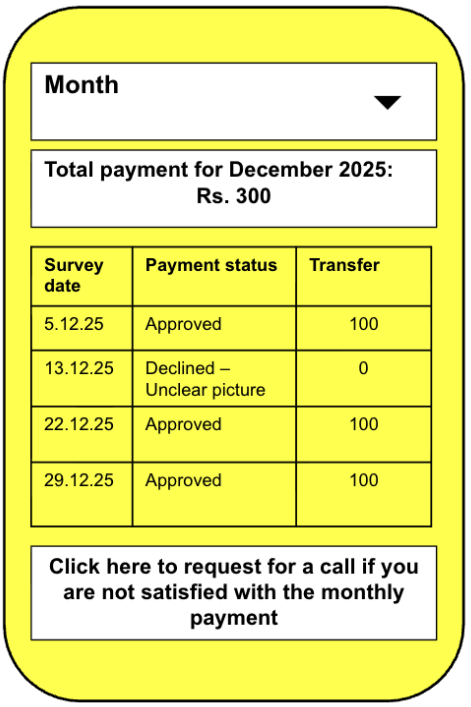
Customer Requirement:

1. DIL:
    a. Customers need the ability for pump operators to be able to view forms approved in the format shown in the right-side image. Transfer amount is fixed for both forms. They should be able to filter the results based on the month when form was filled.
    b. They are going to roll out in AP first and then in Odisha. So they need the display in the local language.
    c. **Out of scope**: Bottom bar requesting for call. Have suggested we can have another card that opens up a grievance form. They are yet to get back.
    d. DIL contract: Link





2. AKRSP wants to print and share forms filled in custom format
3. Goonj wanted users to be able to view their past performance before planning for next quarter.

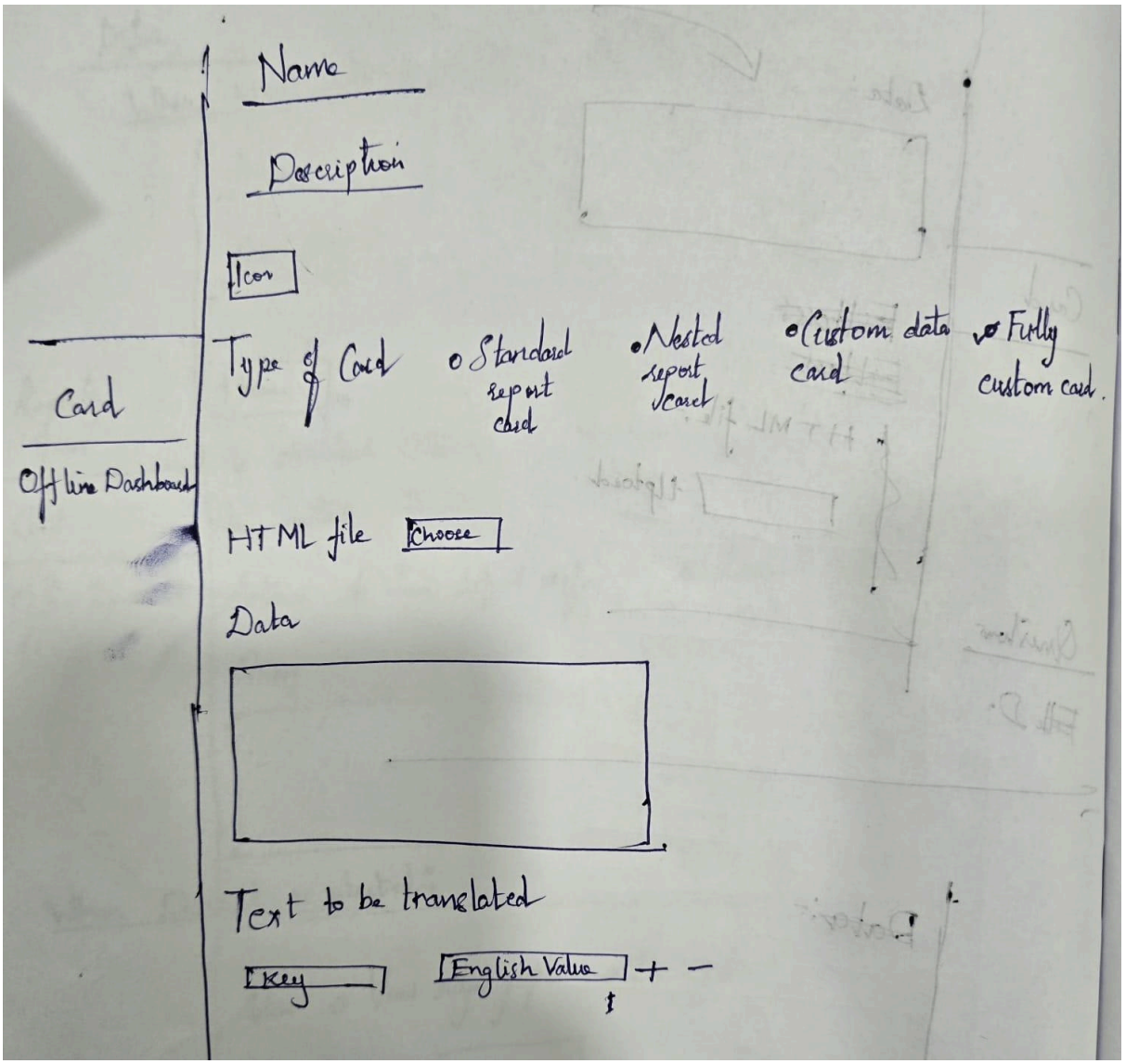
Product requirement:

So far, we have offline cards where data is custom but view is fixed. Now with organisations requiring more simple and custom UI, we are building features that are more generic. Along with it comes the fully custom cards where both data and UI are custom.

Out of scope:

- Supporting in DEA,
- printing media filled part of forms

How the feature works:



1. Avni has a new card type called ‘Fully custom card’
2. When selected, user can upload HTML file.
3. In the Data rule textbox, user can add a JS rule that returns dynamic data to be used when rendering the HTML.
    a. Input to rule: Same as other rules

```
JSON
{
 params: {
     ruleInput: //filters
     db: realmDBObject
  },
 imports: {
 }
```

```
}
```

b. Output from rule:

```JSON
{
 data: {
  }
}
```

c. Needs to run everytime when section filter changes

4. Data to be translated can be defined with English translations which will then display in the translations to be exported as well.
5. So in HTML file, data can be accessed from *data* field and translations from *translations* field within the HTML file.
6. Ability to configure filters at custom card level that will get passed to the data rule.(Missing in above UI)
7. Add the ability to share in whatsapp and print the custom view shown.

Technical details:

Currently, documentation feature stores static HTML and we render it within [AutoHeightWebView](). In our case, data is dynamic.

1. So we can wrap the HTML to make it template literal and pass the data returned from rule and translations to generate the final HTML string.
2. Data rule to use the filters and then return data.
3. HTML is uploaded by the implementer (trusted). It runs on-device, not a server. So, can be considered safe.
4. Can be printed using [react-native-print]()

Example in DIL case:
Configured HTML:

```HTML
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; }
```

```
    body { font-family: sans-serif; background: #FFE500; padding: 12px; }
    .summary {
      background: white;
      border: 2px solid #333;
      border-radius: 6px;
      padding: 12px;
      text-align: center;
      font-weight: 700;
      font-size: 15px;
      margin-bottom: 12px;
    }
    table { width: 100%; border-collapse: collapse; border: 2px solid #333;
}
    th { background: #FFE500; font-weight: 700; padding: 8px; border: 1px
solid #333; font-size: 13px; text-align: left; }
    td { padding: 8px; border: 1px solid #333; font-size: 13px; background:
white; }
  </style>
</head>
<body>

  <div class="summary">
    ${data.total}
  </div>

  <table>
    <thead>
      <tr>
        <th>${translations.surveyDate}</th>
        <th>${translations.paymentStatus}</th>
        <th>${translations.approved}</th>
      </tr>
    </thead>
    <tbody>
      ${data.rows.map(row => `
        <tr>
          <td>${row.surveyDate}</td>
          <td>${translations[row.paymentStatus]}</td>
          <td style="text-align: right;">${row.transfer}</td>
        </tr>
      `).join('')}
    </tbody>
  </table>

</body>
</html>
```

Data rule:

```javascript
(params, imports) => {
return {
  total: 300,
  rows: [
    {
      surveyDate: '5.12.25',
      paymentStatus: 'approved',   // ← key, not display value
      transfer: 100,
    },
    {
      surveyDate: '13.12.25',
      paymentStatus: 'declined',   // ← key
      transfer: 0,
    },
    {
      surveyDate: '22.12.25',
      paymentStatus: 'approved',
      transfer: 100,
    },
    {
      surveyDate: '29.12.25',
      paymentStatus: 'approved',
      transfer: 100,
    },
  ]
}
}
```

Translations:

1. surveyData : Survey Data
2. Similarly…

In client code:

```javascript
const translations = Object.fromEntries(
  Object.entries(config.keys).map(([key, i18nKey]) => [
    key,
    this.I18n.t(i18nKey)
  ])
```

```
);

const buildHTML = (htmlString, data, translations) => {
// evaluate the template literal stored as a string
 const body = new Function('data', 'translations', `return
\`${htmlString}\``)(data, translations);

return `<!DOCTYPE html> <html> <head><meta name="viewport"
content="width=device-width, initial-scale=1.0"></head> <body>${body}</body>
</html>`;
};
```

Render the built HTML using [auto-height webview]

How above (`new Function`) works

`new Function(arg1, arg2, ..., body)` dynamically creates a function at runtime:

```javascript
new Function('data', 'translations', `return
\`${templateHTML}\``)
```

This is equivalent to writing:

```javascript
function anonymous(data, translations) {
  return `<div>${data.total}</div>
          <th>${translations.survey_date}</th>
          ...`
}
```

The backticks around `${templateHTML}` make JS treat the uploaded HTML string as if it were a template literal — so all `${}` inside it get evaluated.

—----------

Discussion with Vinay:

Inputs:
Existing card return value: contract seems be different - can unify?
Test `buildHTML -done`